Tab 1

Improving Education Access and Consistency with AI Driven Facilitation

Mid-Year Technical Report

May Thazin

Department of Software Engineering
The University of Auckland, Auckland, New Zealand

July 24, 2026



Supervisor: Gill Dobbie

Project Partner: Moksh Manglani

Disclaimer *– All these conference papers have been submitted as partial fulfilment for the project requirement for the BE(Hons) degree. Although they have been assessed, no errors or factual information have been corrected or checked.*

Table of Contents

1. INTRODUCTION

This report presents the technical progress of the facilitator decision support system developed during the first half of the project. It describes the current software architecture, learner activity implementation, behavioural data collection pipeline, facilitator dashboard, and the planned machine learning pipeline that will transform learner interaction data into explainable facilitator support recommendations.

2. SYSTEM ARCHITECTURE

2.1 System Overview

2.1 System Components

3. LEARNER ACTIVITY & BEHAVIOUR COLLECTION

3.1 Activity Workflow

3.2 Behavioural Data Collection

3.3 Data Storage

4. BEHAVIOURAL FEATURE ENGINEERING

4.1 Raw Interaction Examples

4.2 Feature Engineering

4.3 Behaviour Dimensions

5. RECOMMENDATION PIPELINE

5.1 Current Implementation

6. MACHINE LEARNING DESIGN

7. CURRENT PROGRESS

8. FUTURE WORK

Problem Statement:

- Facilitators need to understand how individual learners and classes engage with learning activities, but it is difficult to observe every learner consistently. Existing digital learning tools often emphasise outcomes such as scores, accuracy and completion, providing limited information about the behaviours that occurred during the learning process.

- A need to reduce the mental load on the facilitator due to their lack of experience teaching
- 2 Intended users: Facilitator and Student

    - ➢ What information would help a less experienced facilitator teach more effectively?

- Key Advantages of using an AI tool → Impersonal and can be objective with each students data

Your current foundation

Research problem

Facilitators cannot always observe every learner closely, while common educational systems mainly report performance outcomes rather than behavioural trends.

Research question

How can learner interaction data be transformed into explainable individual and class-wide behavioural insights that help facilitators understand learner support needs?

Purpose

To develop and evaluate a decision-support approach that collects learner interaction data, identifies behavioural support trends and communicates them clearly to facilitators.

Intended contribution

An explainable method and prototype for turning learner activity data into facilitator-focused insights, together with evidence about the usefulness and limitations of this approach.

Scope boundary

The system supports facilitator judgement; it does not diagnose learners or make autonomous instructional decisions.

Tab 2

Mid-Year Technical Report

A Facilitator Decision-Support Tool for Year 3–6 Literacy Assessment

Name: May Thazin Supervisor: Gill Dobbie

1. INTRODUCTION

This report presents the current technical progress of a facilitator decision-support tool for tracking literacy learning behaviour in Year 3–6 students. The system delivers short (5–10 minute) reading and comprehension activities to students and presents facilitators with individual and class-level views of student progress. This report focuses only on the technical architecture, implementation status, and remaining engineering work; it does not restate the project's educational motivation or literature review, which are covered in the accompanying research report.

The current proof-of-concept consists of two primary software components: a learner-facing assessment interface and a facilitator-facing dashboard. Student assessment data is captured as structured activity records and processed by a rule-based recommendation engine that generates evidence-linked behavioural indicators for facilitators. While the complete processing pipeline has been implemented, the recommendation thresholds remain provisional and will require calibration using representative learner data during future development.

2. SYSTEM ARCHITECTURE

Student Activity → ActivitySession / StudentResponse → localStorage → Recommendation Engine → Facilitator Dashboard

Fig. 1. Current processing pipeline.

The system follows a pipeline architecture in which raw student interaction data is captured, stored, converted into behavioural indicators by a rule-based recommendation engine, and displayed to facilitators. Figure 1 shows the current pipeline; component status is summarised in Table I.

Table 1. Component status

Component	Responsibility	Status
Student activity interface	Presents reading passage, comprehension questions, and reflection prompts	Implemented (limited question set)
Event/response logger	Captures student interaction data as structured response records	Implemented
Local data store	Persists roster, activity, and response records (see 2.1)	Implemented (browser localStorage)
Recommendation engine	Converts response data into behavioural indicators and facilitator guidance using rule-based logic	Implemented (thresholds are provisional — see 4.1)
Student-history aggregation	Groups repeated activity records to display changes across sessions	Partially implemented
Class aggregation	Summarises activity results across students	Implemented at overview level
Facilitator dashboard	Displays student cards, individual trends, and class trends	Implemented

The architecture currently allows student response data to flow end-to-end from activity completion to a generated facilitator recommendation, without manual intervention. The primary remaining engineering work is calibrating the rule thresholds and migrating storage to a proper backend and database (Section 7), rather than building the core pipeline itself.

2.1 Implementation Technologies

The current implementation uses the following stack:

- Framework: Next.js 16, using the App Router
- Language: TypeScript
- UI: React with Tailwind CSS
- Data model: TypeScript types for ActivitySession, StudentResponse, QuestionResponse, FacilitatorRecommendation, Roster, ClassRecord, and StudentRecord

- Recommendation logic: a rule-based TypeScript engine, run directly against stored response data
- Storage: browser localStorage (no backend or database yet)

Storing data in browser localStorage was a deliberate MVP simplification, allowing the activity, data model, and recommendation logic to be built and tested quickly without the added complexity of a backend service. This choice has known limitations that are treated as current constraints rather than oversights: data is scoped to a single browser and device, so a facilitator's roster and class data are not currently accessible from another computer or after browser data is cleared, and there is no support yet for multiple facilitators sharing the same class data. Migrating to a backend with persistent database storage is planned work, described in Section 7, once the recommendation logic has stabilised.

Activity data is stored as structured JSON objects representing completed assessment sessions. This allows learner responses to be reprocessed as recommendation logic evolves without requiring activities to be repeated, simplifying future refinement of the behavioural analysis pipeline.

2.2 Operational Assumptions and Constraints

The current proof-of-concept is based on several operational assumptions. Students are assumed to have access to a digital device for a short 5–10 minute assessment before each class. Repeated assessments are intended to provide a sequence of activity records that can later be used to identify changes in learner interaction patterns over time.

The current activity domain is limited to reading comprehension. Each assessment presents a short passage followed by questions that test both overall understanding and interpretation of the task. The initial prototype contains one main-idea question and one supporting-detail question.

These assumptions affect the technical design. Repeated use requires each session to be associated with a student, class, activity, and timestamp. Multi-student use also requires shared persistent storage so that assessments completed on separate devices can be accessed through one facilitator dashboard. The current *localStorage* implementation supports single-browser prototyping but does not yet support this intended deployment model.

2.3 Learner Activity Design

The learner-facing component is designed as a short digital assessment intended to be completed in approximately 5–10 minutes before each class. Each completed assessment generates a timestamped activity record that contributes to an individual learner history, allowing repeated sessions to be analysed over time.

The current proof-of-concept is limited to reading comprehension and consists of a single passage followed by two multiple-choice questions. The questions assess two comprehension categories:

- Main idea – identifying the central idea of the passage.
- Supporting detail – selecting evidence that best supports the main idea.

Each question records the learner's response, correctness, response time, confidence rating, and associated interaction measurements. These values are combined with session-level information, such as total completion time, passage revisits, answer revisions, and hint usage, to produce the feature vector used by the recommendation engine.

The current activity serves as a controlled proof-of-concept for the analysis pipeline. Additional passages, question types, and subject domains can be incorporated by extending the activity definition without modifying the overall processing architecture. Each completed assessment produces one *ActivitySession* record that forms the input to the behavioural analysis pipeline described in Section 4.

3. INTERACTION EVENT MODEL

Student interactions are captured as discrete, timestamped events, grouped by student and activity. This structure was chosen so that raw event history is preserved and can be reprocessed as categorisation logic is developed, without needing to re-collect data.

Table 2. Event types currently captured

Event type	Description	Status
ACTIVITY_STARTED	Marks the beginning of a student session	Implemented
ACTIVITY_COMPLETED	Marks the end of a student session	Implemented
QUESTION_OPENED	Records when a question is displayed to the student	Implemented
ANSWER_SUBMITTED	Records the student's final response to a question	Implemented
[Additional event types, e.g. ANSWER_CHANGED, HINT_OPENED]	To be added as the question set and interaction features expand	Planned

Events are ordered by timestamp before any downstream processing, and activity boundaries are defined by the ACTIVITY_STARTED and ACTIVITY_COMPLETED events. At present, the system displays this raw event data directly to facilitators without further transformation; Section 4 describes how this data is intended to be interpreted. Preserving raw interaction events allows recommendation logic to evolve over time without requiring learner activities to be repeated, improving reproducibility and supporting future experimentation.

4. BEHAVIOURAL CATEGORISATION LOGIC

Raw student response data is converted into facilitator-facing indicators by a rule-based recommendation engine, implemented directly against the StudentResponse and QuestionResponse data model. It is important to be precise about what these indicators represent: they are not intended to diagnose a student's underlying learning needs or fixed learning style. Rather, each indicator flags an observable interaction pattern for a facilitator to review and interpret using their own professional judgement. This framing is deliberate, since a small set of interaction features cannot reliably determine a child's true learning needs; it can only surface patterns that may be relevant to them.

4.1 Implemented indicators

The recommendation engine evaluates the extracted feature set against six deterministic rule groups. Each triggered rule generates a behavioural indicator together with the evidence responsible for triggering that indicator, allowing every recommendation to be traced directly back to observed learner interactions. The initial rule set was selected to demonstrate a representative range of observable learner behaviours, including performance, confidence, engagement, and interaction patterns. Together, these rules provide sufficient coverage to validate the recommendation pipeline while remaining intentionally small for this proof-of-concept.

- Task-understanding concern — triggered by slow first-answer delay, repeated answer changes, repeated passage revisits, or long per-question time.
- Accuracy concern — triggered whenever accuracy is below 100%, with severity increasing below 50%.
- Hints used — triggered whenever one or more hints were opened.
- Low answer confidence — triggered when one or more answers were marked as low self-reported confidence.
- High perceived difficulty — triggered by the learner's own reflection rating.
- Extended time on task — triggered when total activity time exceeds a fixed duration.

If none of these conditions are met, the engine returns a default 'secure progress' indicator, so that every completed activity produces a result rather than an empty one. Indicators, evidence, and an overall summary/action pair are compiled into a single FacilitatorRecommendation record per student per activity, which is what the dashboard displays.

4.2 A note on thresholds

The numeric thresholds used above (for example, accuracy below 0.75, a first-answer delay of 20 seconds or more, or a perceived-difficulty rating of 3 or higher) are current engineering placeholders, set to reasonable-seeming values for MVP development rather than derived from research or real student data. This is an explicit and acknowledged limitation: the rule logic and output structure are implemented and working, but the specific cut-off values require calibration once real or research-informed data is available. This distinction; working logic versus tuned thresholds, is treated separately throughout this report.

5. WORKED EXAMPLE (SYSTEM-GENERATED)

The analysis approach is rule-based rather than machine-learning based, since it does not require a labelled dataset and produces results that can be directly traced back to the underlying response values — an important property given that a labelled dataset for this population does not yet exist.

The following example was produced by running the implemented recommendation engine against a real activity response. A student ('Student 1') completed a two-question reading activity, taking 155 seconds in total, requesting one hint, and changing an answer twice on the second question. The engine's actual output is shown below, in plain-English form.

5.1 Input summary

- Question 1 (main idea): answered correctly, after one hint, first interaction at 18s, one passage revisit.
- Question 2 (supporting detail): answered incorrectly, with two answer changes, first interaction at 24s, one passage revisit.
- Total time on task: 155 seconds. Perceived difficulty: 2/5. Reflection: "I found the second question tricky."

5.2 Generated recommendation

Summary: Review how the student interpreted the task before reteaching content.

Suggested action: Ask the student to explain what the question was asking and how they decided where to look in the passage.

Table 3. General indicators

Indicator	Severity	Evidence
Possible task-understanding concern	High	First answer delay averaged 21s; passage revisits: 2; answer changes: 2.
Accuracy concern	Medium	1 of 2 answers were correct.
Hint used	Low	The student opened 1 hint.
Low answer confidence	Medium	1 answer had confidence marked as not sure or guessing.

This example demonstrates that the pipeline runs end-to-end on real input: a completed activity response produces a structured, evidence-linked recommendation without manual intervention. It also illustrates the current threshold sensitivity discussed in Section 4.2: for instance, the 'task-understanding concern' indicator was assigned high severity primarily because of the two answer changes and repeated passage revisits, which may or may not warrant that severity level once thresholds are calibrated against real data.

6. IMPLEMENTED FUNCTIONALITY

The following have been implemented and are running as working software:

- A student-facing web activity, currently presenting a small set of comprehension questions within the 5–10 minute target window.
- Structured logging of student responses, including per-question timing, hint use, answer changes, passage revisits, and confidence.
- A rule-based recommendation engine that computes accuracy, hint, revision, timing, and confidence measures directly from stored response data and generates a structured, evidence-linked recommendation per completed activity (Section 4, Section 5).
- A facilitator-facing dashboard displaying individual student cards, generated recommendations, and a class-level overview.

The following remain limitations or planned work rather than completed features:

- Rule thresholds are provisional engineering placeholders and have not been calibrated against real or research-informed data (Section 4.2).
- Data is currently stored in browser localStorage only; there is no backend or database, so data does not persist across devices or support multiple facilitators (Section 2.1).

- Any trained machine-learning component. No labelled dataset currently exists, so this remains future work rather than near-term work (see Section 7).

This scope reflects a deliberate technical decision to prioritise a complete, working pipeline — from student activity through to a generated facilitator recommendation — ahead of production-grade storage or calibrated thresholds, so that remaining work can be validated against a real system rather than a design document.

6.1 Technical Success Criteria

The current proof-of-concept is evaluated against a set of technical success criteria that assess whether the proposed learner analytics framework functions as intended. The technical success criteria are organised around the four major stages of the system: data collection, learner analysis, recommendation generation, and facilitator decision support. The evaluation focuses on validating the technical feasibility of the end-to-end software pipeline rather than the educational effectiveness of the recommendations, which remains future work.

Table 4. Success criteria table

Criterion	Success Condition	Current Status
Learner Data Collection	All required learner interaction metrics (e.g., timing, hints, confidence, revisions, accuracy) are correctly captured and stored.	Achieved
Behavioural Analysis	Learner interaction data is correctly transformed into behavioural indicators using the implemented rule-based engine.	Achieved
Recommendation Generation	Evidence-linked facilitator recommendations are generated for completed learner activities.	Achieved
Recommendation Explainability	Every recommendation includes the learner interactions that triggered it, ensuring outputs are transparent and traceable.	Achieved
Facilitator Decision Support	Individual and class-level recommendations are presented through the dashboard to support facilitator judgement.	Achieved
End-to-End Functionality	The complete pipeline operates successfully from learner activity completion through to facilitator recommendation generation.	Achieved
Persistent Multi-Device Storage	Learner data is shared across devices through a backend database.	Planned
Calibrated Recommendation Thresholds	Rule thresholds are validated using representative learner data.	Planned

Collectively, these criteria demonstrate that the proposed learner analytics framework operates as a complete end-to-end software system. The remaining work focuses primarily on improving deployment infrastructure and refining recommendation quality, rather than implementing additional core pipeline functionality.

7. REMAINING WORK AND PLANNING

Table 5. Planned implementation timeline

Milestone	Description
Threshold review	Review and adjust the provisional thresholds in Section 4.2 against pilot or research-informed data
Backend and database migration	Replace browser localStorage with a persistent backend and database, enabling multi-device and multi-facilitator use
Expanded question set	Increase question count/types to support question-type accuracy breakdowns
Pilot data collection	Collect a small real or synthetic dataset to sanity-check rule behaviour and threshold sensitivity across a range of students
Labelled dataset & ML comparison	Longer-term: construct a labelled dataset and evaluate a Decision Tree / Random Forest baseline against the rule-based system

Machine-learning comparison is retained as a longer-term goal rather than near-term work, since it depends on a labelled dataset that does not yet exist and cannot be responsibly substituted with synthetic labels at this stage.

8. CONCLUSION

The project currently has a working end-to-end pipeline: a student-facing literacy activity captures structured response data, a rule-based recommendation engine converts that data into evidence-linked behavioural indicators, and a facilitator dashboard presents individual and class-level views of the results. This was demonstrated in Section 5 using a real system-generated example. The main acknowledged limitations are that rule thresholds are currently provisional engineering placeholders rather than calibrated values, and that data is stored only in browser localStorage pending a backend and database migration. Both are treated as scoped, near-term engineering work rather than unresolved design questions, and are addressed in the implementation plan in Section 7. Overall, the project demonstrates the technical feasibility of transforming learner interaction data into explainable, facilitator-focused recommendations, providing a foundation for future validation and comparison with machine-learning approaches. This establishes a practical engineering baseline upon which future AI-assisted recommendation techniques can be evaluated.